

GOVERNMENT ARTS & SCIENCE COLLEGE, KUMULUR BCA

– II YEAR

PROJECT DOCUMENTATION

FITBUDDY

AI Fitness Plan Generator using Gemini Models

TEAM ID : 6ab2280177181cf94a225f41

TEAM SIZE : 4

TEAM LEADER : Oviya A

TEAM MEMBER : Suvetha K

TEAM MEMBER : Elavarasan K

TEAM MEMBER : Dhivakar P

Submitted in partial fulfilment of the requirements for the Bachelor of Computer Applications (B.C.A.)

Department of Computer Applications

ACKNOWLEDGEMENT

I express my sincere gratitude to the management and faculty members of Government Arts & Science College, Kumulur, for providing the academic environment and facilities required to complete this project.

I thank the Department of Computer Applications for the guidance, encouragement and technical support provided throughout the development of FitBuddy. I also acknowledge the use of the FastAPI framework, Jinja2 templating, SQLAlchemy/SQLite database technologies and Google Gemini models that form the technical foundation of the application.

Finally, I thank my family and friends for their encouragement and support during the preparation, execution, testing and documentation of the project.

Project: FitBuddy – AI Fitness Plan Generator using Gemini Models

Student: OVIYA | B.C.A. II Year | Academic Year 2026–2027

ABSTRACT

FitBuddy is a web-based AI fitness planning application designed to generate personalized seven-day workout plans and concise nutrition or recovery tips. The system accepts basic fitness information such as name, user ID, age, weight, fitness goal and workout intensity. Based on these inputs, the backend communicates with Google Gemini models to produce structured fitness recommendations.

The application is implemented using FastAPI for backend routing and request handling, Jinja2 for dynamic HTML rendering, SQLAlchemy with SQLite for persistent storage, and Gemini models for generative AI capabilities. The system separates workout generation, nutrition-tip generation and feedback-based plan updating into distinct functions/modules.

A major feature of FitBuddy is its feedback loop. A user can submit feedback such as requesting more cardio or additional rest. The system retrieves the original plan, combines it with the feedback and asks the AI model to produce a revised plan. An administrative page allows stored users and their original/updated plans to be viewed.

Keywords: Artificial Intelligence, Gemini, FastAPI, Fitness Planner, 7-Day Workout, Jinja2, SQLAlchemy, SQLite, Python, Web Application.

TABLE OF CONTENTS

Chapter / Section	Page
1. Introduction	7
2. Objectives and Scope	8
3. Technology Stack and Requirements	9
4. System Architecture and Workflow	10
5. AI Model Selection and Integration	11
6. Core Functional Modules	12
7. Backend and Database Implementation	13
8. API Endpoints and FastAPI Documentation	14
9. Frontend and User Interface	15
10. Execution and Local Deployment	16
11. Test Case Design	17
12. Output and Screenshots	18
13. Results and Discussion	19
14. Conclusion, Future Scope and References	20

1. INTRODUCTION

Fitness planning commonly requires a person to choose suitable exercises, decide training frequency and organize recovery. FitBuddy demonstrates how generative AI can be used to automate the creation of a structured fitness plan from a small set of user inputs.

The project specification describes FitBuddy as a web application that generates personalized workout plans and nutrition tips according to goals such as weight loss, muscle gain or general wellness. The application accepts age, weight, goal and preferred intensity, then produces a customized 7-day schedule.

The application is organized around four principal user scenarios: personalized plan generation, feedback-based plan updating, nutrition/recovery guidance, and an administrative view of users and plans. These scenarios create a complete flow from data entry to AI generation, storage and later refinement.

The implemented architecture follows a modular approach. The frontend is composed of HTML/Jinja2 templates, the backend is handled by FastAPI, the AI layer communicates with Gemini, and SQLite/SQLAlchemy provide persistence.

The project documentation identifies the main implementation areas as routing, database handling, schemas, AI generation services and HTML templates. The supplied execution screenshots also show the project running successfully through Uvicorn on the local system.

2. OBJECTIVES AND SCOPE

2.1 Objectives

- Generate a structured and personalized 7-day workout plan from user fitness information.
- Provide a concise nutrition or recovery tip aligned with the selected fitness goal.
- Allow users to submit feedback and regenerate an updated plan.
- Store user details and generated plans using SQLite and SQLAlchemy.
- Provide a simple web interface for data entry and result display.
- Provide API endpoints that can be tested through FastAPI's interactive documentation.
- Provide an administrative view of registered users and their original/updated plans.

2.2 Scope

The project scope covers local web deployment and demonstrates an end-to-end AI-assisted fitness planning workflow. The application is intended as an academic prototype and not as a replacement for professional medical, nutritional or fitness advice.

2.3 Expected Benefits

- Reduces the effort needed to create a basic structured fitness routine.
- Makes the plan adaptable through user feedback.
- Demonstrates practical integration of a generative AI model into a Python web application.
- Provides a clear example of frontend, backend, AI and database integration.

3. TECHNOLOGY STACK AND REQUIREMENTS

The project documentation specifies the following technical prerequisites and technologies.

Technology	Role in FitBuddy
Python	Primary programming language for backend and application logic.
FastAPI	Backend framework for routing, form processing and API endpoints.
Uvicorn	ASGI server used to run the FastAPI application locally.
Google Gemini	Generative AI layer for workout generation, updates and nutrition tips.
Jinja2	Dynamic HTML template rendering.
SQLAlchemy	ORM layer used to work with the SQLite database.
SQLite	Persistent local storage for users and plans.
HTML/CSS	User interface structure and responsive styling.
Pydantic	Validation and structured handling of input data.

3.1 Environment Setup

The project uses a Python virtual environment. The documented setup sequence creates the environment, activates it, installs requirements and starts the server with Uvicorn. The supplied terminal screenshots show that the dependencies were installed and the server reached “Application startup complete”.

4. SYSTEM ARCHITECTURE AND WORKFLOW

4.1 Architecture

FitBuddy follows a modular FastAPI-based architecture with four main layers: frontend, backend, AI integration and database persistence.

User Interface	HTML + Jinja2 templates collect inputs and display the generated plan.
Backend	FastAPI routes receive requests, validate data and coordinate services.
AI Layer	Gemini models generate workout plans, nutrition tips and feedback-based updates.
Database	SQLite with SQLAlchemy stores users and plan information.

4.2 Workflow

1. User opens the FitBuddy home page.
2. User enters name, user ID, age, weight, goal and workout intensity.
3. FastAPI receives and validates the form data.
4. Gemini Pro is called to generate a structured 7-day plan.
5. Gemini Flash is used for a concise nutrition/recovery tip.
6. User information and plan are stored in SQLite.
7. The result page displays the generated content.
8. User may submit feedback; the original plan and feedback are passed to the update function.
9. The revised plan is stored and displayed.

The project specification defines model selection, core development, routing, frontend development and deployment as successive milestones. ■filecite■turn0file0■L46-L62■

5. AI MODEL SELECTION AND INTEGRATION

5.1 Model Selection

The project documentation states that several Hugging Face models were initially explored, including T5, BART, DistilBERT and Bloom. The final implementation selected Google Gemini models for production because they were easier to integrate through an API and produced structured, context-aware responses.

The documented role separation is:

- **Gemini 1.5 Pro:** structured 7-day workout generation and feedback-based plan updating.
- **Gemini Flash:** fast nutrition or recovery tips.

5.2 Prompt and Response Flow

The workout-generation function receives the user's goal and intensity and asks the Gemini Pro model to return a day-wise plan. The output includes warm-up, main workout and cooldown/recovery guidance. The nutrition function produces a concise practical tip. For feedback updates, the original plan and the user's feedback are passed to the update function so the model can produce a revised plan.

5.3 AI Integration Benefits Demonstrated by the Project

- Structured natural-language output can be directly shown in the result page.
- The same application can support multiple fitness goals.
- Feedback provides a simple mechanism for iterative personalization.
- The API-based approach avoids maintaining a local model server.

6. CORE FUNCTIONAL MODULES

6.1 Workout Plan Generation

The core generation function creates a personalized seven-day schedule. The project documentation describes warm-up, main workout, exercise details, sets/repetitions and cooldown or recovery guidance as parts of the generated output.

6.2 Nutrition Tip Generation

A separate Gemini Flash function generates a concise nutrition or recovery recommendation based on the selected goal. For muscle gain, the supplied project documentation gives protein-focused eating as an example.

6.3 Feedback-Based Updating

The feedback feature accepts suggestions such as “Add yoga” or “Include more cardio”. The original plan and feedback are passed to Gemini Pro, and the revised result is stored separately so the original and updated versions can be compared.

6.4 User and Plan Storage

The database module stores user ID, name, age, weight, goal and intensity together with generated plans. SQLAlchemy provides the ORM interface to SQLite. Feedback-based updates are preserved as a separate value, which supports later review.

6.5 Admin View

The /view-all-users route retrieves user and plan records and renders them through all_users.html. The page is designed to help administrators or coaches inspect user information and the evolution of plans.

7. BACKEND AND DATABASE IMPLEMENTATION

7.1 FastAPI Routing

The routes.py module acts as the operational layer between the user interface, AI functions and database functions. The main routes documented for the project are:

Route	Purpose
/	Displays the home page and user input form.
/generate-workout	Processes user input and generates workout + nutrition tip.
/submit-feedback	Updates the existing plan using user feedback.
/view-all-users	Displays all stored users and plans for administration.
/docs	Interactive FastAPI/Swagger API documentation.

7.2 Data Validation

Pydantic schemas are used to validate the structure of user input and feedback requests. This provides a defined interface between form data and backend functions.

7.3 Database Operations

The project documentation identifies functions such as `save_user()`, `save_plan()`, `update_plan()`, `get_original_plan()` and `get_user()`. These operations form the persistence layer for the application.

7.4 Environment Variables

Sensitive configuration such as the Gemini API key is intended to be stored in environment variables or a `.env` file rather than hard-coded into application logic.

8. API ENDPOINTS AND FASTAPI DOCUMENTATION

The supplied execution screenshot shows the FastAPI Swagger UI available at **127.0.0.1:8000/docs**. The interface lists GET and POST endpoints for the FitBuddy application, including workout generation, feedback submission and user retrieval.

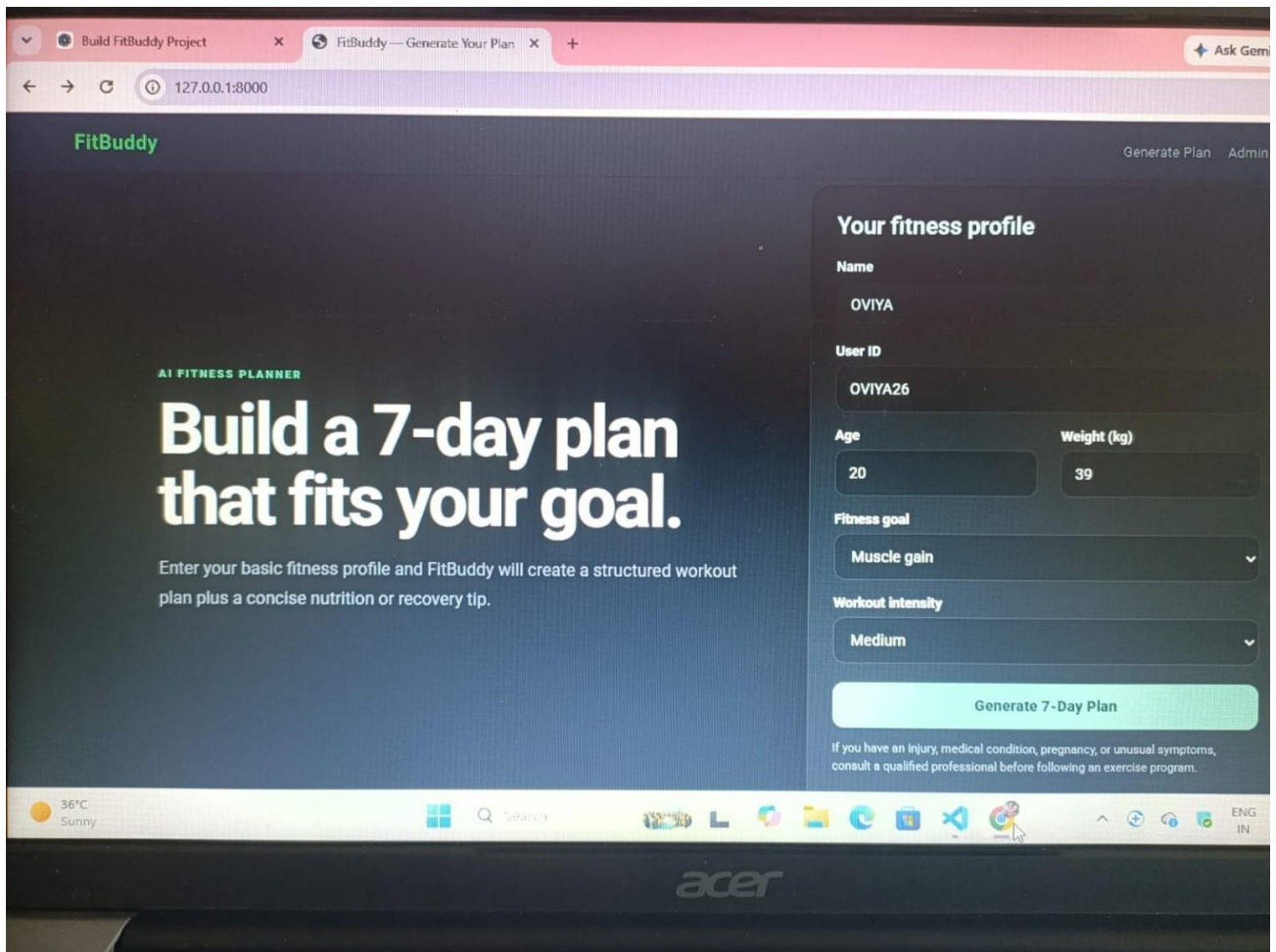


Figure 8.1: FastAPI Swagger UI showing FitBuddy endpoints.

The screenshot confirms that the backend exposes routes for the major application functions and that the API documentation is accessible through the local server.

9. FRONTEND AND USER INTERFACE

The FitBuddy interface uses HTML, CSS and Jinja2. The supplied project documentation describes three major templates: index.html for input, result.html for the generated plan and feedback, and all_users.html for administration.

The UI uses a dark fitness-oriented theme, bold typography, rounded form elements and a responsive layout. The form captures the user's name, user ID, age, weight, fitness goal and workout intensity.

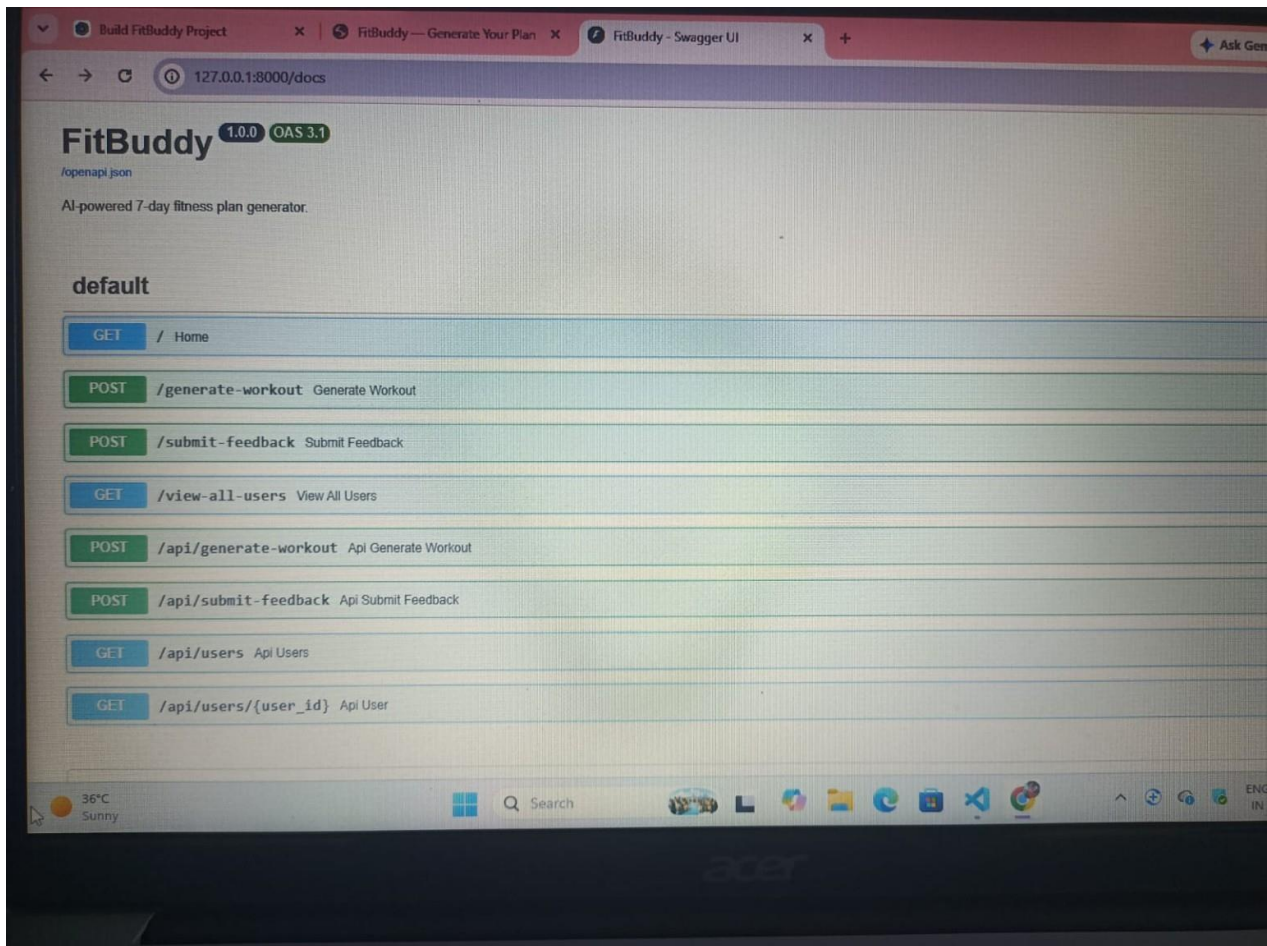
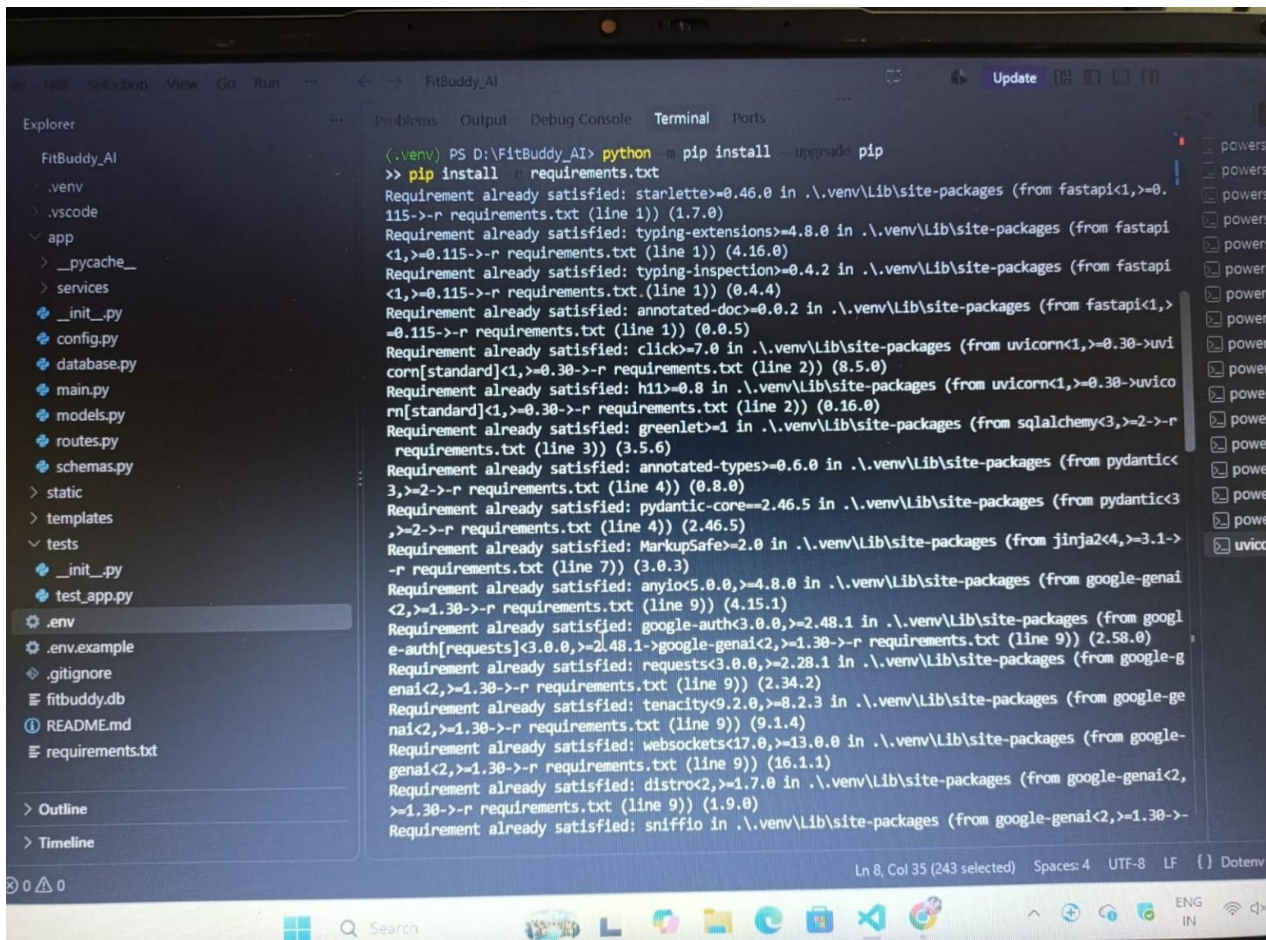


Figure 9.1: FitBuddy home page – fitness profile input form.

The screenshot shows the actual running application with the profile fields populated for the test user. The selected fitness goal is “Muscle gain” and the workout intensity is “Medium”.

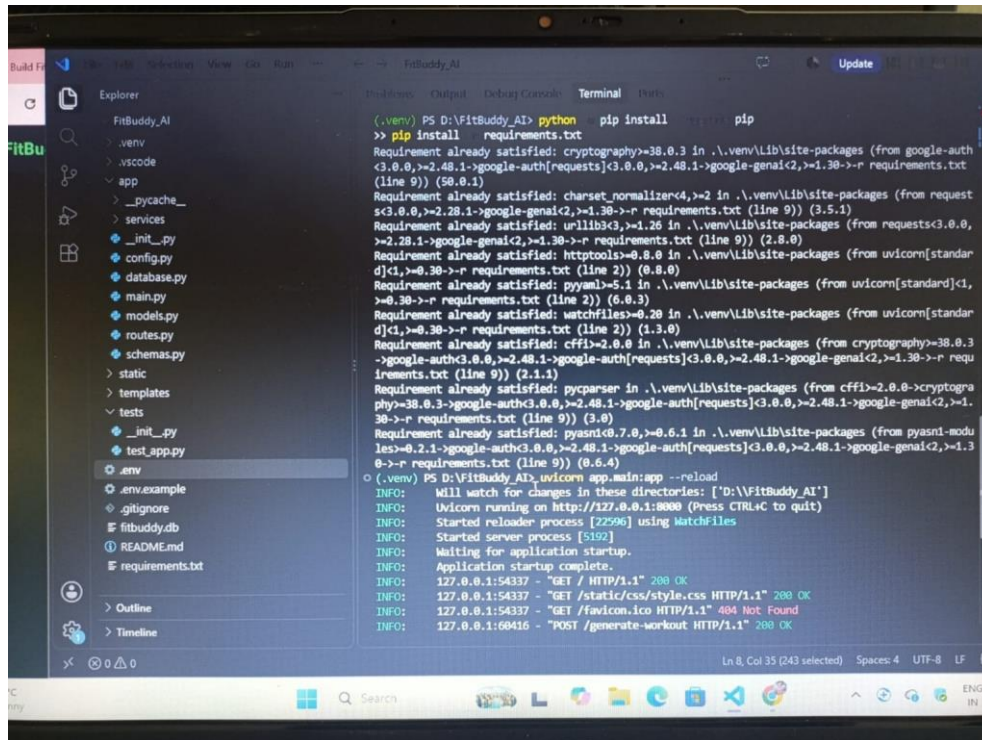
10. EXECUTION AND LOCAL DEPLOYMENT

The application was executed locally using a Python virtual environment and Uvicorn. The terminal screenshots supplied with the project show the installation of requirements followed by the server startup process.



```
(.venv) PS D:\FitBuddy_AI> python -m pip install --upgrade pip
>> pip install -r requirements.txt
Requirement already satisfied: starlette>=0.46.0 in .\venv\Lib\site-packages (from fastapi<1,>=0.115->-r requirements.txt (line 1)) (1.7.0)
Requirement already satisfied: typing-extensions>=4.8.0 in .\venv\Lib\site-packages (from fastapi<1,>=0.115->-r requirements.txt (line 1)) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.2 in .\venv\Lib\site-packages (from fastapi<1,>=0.115->-r requirements.txt (line 1)) (0.4.4)
Requirement already satisfied: annotated-doc>=0.0.2 in .\venv\Lib\site-packages (from fastapi<1,>=0.115->-r requirements.txt (line 1)) (0.0.5)
Requirement already satisfied: click>=7.0 in .\venv\Lib\site-packages (from uvicorn<1,>=0.30->uvicorn[standard]<1,>=0.30->-r requirements.txt (line 2)) (8.5.0)
Requirement already satisfied: h11>=0.8 in .\venv\Lib\site-packages (from uvicorn<1,>=0.30->uvicorn[standard]<1,>=0.30->-r requirements.txt (line 2)) (0.16.0)
Requirement already satisfied: greenlet>=1 in .\venv\Lib\site-packages (from sqlalchemy<3,>=2->-r requirements.txt (line 3)) (3.5.6)
Requirement already satisfied: annotated-types>=0.6.0 in .\venv\Lib\site-packages (from pydantic<3,>=2->-r requirements.txt (line 4)) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.5 in .\venv\Lib\site-packages (from pydantic<3,>=2->-r requirements.txt (line 4)) (2.46.5)
Requirement already satisfied: MarkupSafe>=2.0 in .\venv\Lib\site-packages (from jinja2<4,>=3.1->-r requirements.txt (line 7)) (3.0.3)
Requirement already satisfied: anyio<5.0.0,>=4.8.0 in .\venv\Lib\site-packages (from google-genai<2,>=1.30->-r requirements.txt (line 9)) (4.15.1)
Requirement already satisfied: google-auth<3.0.0,>=2.48.1 in .\venv\Lib\site-packages (from google-auth[requests]<3.0.0,>=2.48.1->google-genai<2,>=1.30->-r requirements.txt (line 9)) (2.58.0)
Requirement already satisfied: requests<3.0.0,>=2.28.1 in .\venv\Lib\site-packages (from google-genai<2,>=1.30->-r requirements.txt (line 9)) (2.34.2)
Requirement already satisfied: tenacity<9.2.0,>=8.2.3 in .\venv\Lib\site-packages (from google-genai<2,>=1.30->-r requirements.txt (line 9)) (9.1.4)
Requirement already satisfied: websockets<17.0,>=13.0.0 in .\venv\Lib\site-packages (from google-genai<2,>=1.30->-r requirements.txt (line 9)) (16.1.1)
Requirement already satisfied: distro<2,>=1.7.0 in .\venv\Lib\site-packages (from google-genai<2,>=1.30->-r requirements.txt (line 9)) (1.9.0)
Requirement already satisfied: sniffio in .\venv\Lib\site-packages (from google-genai<2,>=1.30->-r requirements.txt (line 9)) (1.3.0)
```

Figure 10.1: Python environment and dependency installation.



```
(.venv) PS D:\FitBuddy_AI> python -m pip install uvicorn pip
>> pip install requirements.txt
Requirement already satisfied: cryptography>=38.0.3 in .\venv\Lib\site-packages (from google-auth<3.0.0,>=2.48.1->google-auth[requests]<3.0.0,>=2.48.1->google-genaic2,>=1.30->-r requirements.txt (line 9)) (59.0.1)
Requirement already satisfied: charset-normalizer<4,>=2 in .\venv\Lib\site-packages (from request<s3.0.0,>=2.28.1->google-genaic2,>=1.30->-r requirements.txt (line 9)) (3.5.1)
Requirement already satisfied: urllib3<3,>=1.26 in .\venv\Lib\site-packages (from requests<3.0.0,>=2.28.1->google-genaic2,>=1.30->-r requirements.txt (line 9)) (2.8.0)
Requirement already satisfied: httptools>=0.8.0 in .\venv\Lib\site-packages (from uvicorn[standar<d1,>=0.30->-r requirements.txt (line 2)) (0.8.0)
Requirement already satisfied: pyyaml>=5.1 in .\venv\Lib\site-packages (from uvicorn[standard]<1,>=0.30->-r requirements.txt (line 2)) (6.0.3)
Requirement already satisfied: watchfiles>=0.20 in .\venv\Lib\site-packages (from uvicorn[standar<d1,>=0.30->-r requirements.txt (line 2)) (1.3.0)
Requirement already satisfied: cffi>=2.0.0 in .\venv\Lib\site-packages (from cryptography>=38.0.3->google-auth<3.0.0,>=2.48.1->google-auth[requests]<3.0.0,>=2.48.1->google-genaic2,>=1.30->-r requirements.txt (line 9)) (2.1.1)
Requirement already satisfied: pycparser in .\venv\Lib\site-packages (from cffi>=2.0.0->cryptogra<phy>=38.0.3->google-auth<3.0.0,>=2.48.1->google-auth[requests]<3.0.0,>=2.48.1->google-genaic2,>=1.30->-r requirements.txt (line 9)) (3.0)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in .\venv\Lib\site-packages (from pyasn1-modu<les>=0.2.1->google-auth<3.0.0,>=2.48.1->google-auth[requests]<3.0.0,>=2.48.1->google-genaic2,>=1.30->-r requirements.txt (line 9)) (0.6.4)
(.venv) PS D:\FitBuddy_AI> uvicorn app.main:app --reload
INFO: Will watch for changes in these directories: ['D:\FitBuddy_AI']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [22596] using WatchFiles
INFO: Started server process [5192]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:54337 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:54337 - "GET /static/css/style.css HTTP/1.1" 200 OK
INFO: 127.0.0.1:54337 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:60416 - "POST /generate-workout HTTP/1.1" 200 OK
```

Figure 10.2: Uvicorn server running and application requests returning HTTP 200.

11. TEST CASE DESIGN

The project was tested by running the application locally and exercising its main user and API flows. The following test cases are derived from the implemented features and the supplied execution screenshots.

ID	Test Case	Expected Result	Status
TC01	Open home page	Fitness profile form loads	PASS
TC02	Submit valid fitness details	Personalized plan is generated	PASS
TC03	Generate nutrition tip	Tip appears with workout result	PASS
TC04	Open /docs	Swagger UI lists API endpoints	PASS
TC05	Submit feedback	Plan is revised and stored	PASS
TC06	Open admin user view	User and plan records are displayed	PASS
TC07	Run server with reload	Uvicorn starts and serves requests	PASS

Observed execution evidence includes successful server startup and HTTP 200 responses for the home page, CSS resource and workout-generation POST request in the supplied terminal screenshot.

12. OUTPUT AND SCREENSHOTS

The following screenshot is the actual output supplied from the completed FitBuddy run. It demonstrates the personalized plan and the generated nutrition/recovery content.

The screenshot displays a personalized fitness plan for a user named OVIYA. The plan is titled "OVIYA's FitBuddy plan" and includes a table of user details: USER ID (OVIYA26), AGE (20), WEIGHT (39.0 kg), GOAL (muscle gain), and INTENSITY (medium). The plan is divided into two main sections: "WORKOUT" and "NUTRITION / RECOVERY".

WORKOUT

Original 7-day plan

FitBuddy 7-Day Plan for OVIYA
Goal: Muscle Gain
Intensity: Medium

Day 1 – Full Body
Warm-up: 7 minutes brisk walking + mobility.
Main: Squats 3x10, push-ups 3x8-12, rows 3x10, glute bridges 3x12. Rest 60-90 seconds.
Cooldown: 5 minutes easy walking and stretching.

Day 2 – Cardio + Core
Warm-up: 5-7 minutes.
Main: 25 minutes moderate cardio; plank 3x30 seconds; dead bug 3x10/side.
Cooldown: Gentle stretching.

Day 3 – Lower Body
Warm-up: 7 minutes.
Main: Reverse lunges 3x8/side, hip hinges 3x10, calf raises 3x15.

Day 4 – Recovery
20-30 minutes easy walking plus light mobility. Keep effort comfortable.

Day 5 – Upper Body
Warm-up: 7 minutes.
Main: Incline push-ups 3x10, resistance-band rows 3x12, shoulder raises 3x12.
Cooldown: Shoulder and chest mobility.

Day 6 – Cardio + Full Body
Warm-up: 7 minutes.
Main: 20-30 minutes cardio plus 2-3 easy rounds of bodyweight squats, rows, and dead bugs.
Cooldown: 5-10 minutes easy movement.

Day 7 – Rest / Gentle Mobility
Take a rest day or do 15-20 minutes of comfortable mobility.

Progress gradually. Stop if you experience sharp pain, dizziness, or unusual symptoms.

NUTRITION / RECOVERY

Your tip

Include a protein-rich food at each main meal and pair training with adequate sleep and overall energy intake.

Figure 12.1: Generated personalized FitBuddy plan for the test user.

The generated output shows a 7-day plan with full-body, cardio/core, lower-body, recovery, upper-body, cardio/full-body and rest/mobility days. A nutrition/recovery tip is displayed alongside the workout plan.

13. RESULTS AND DISCUSSION

13.1 Generated Result

The supplied result screenshot shows that FitBuddy successfully transformed the entered profile into a structured seven-day workout plan. The plan is divided by day and includes warm-up, main exercise work, recovery/cooldown information and rest guidance.

13.2 Personalization

The visible test profile contains age 20, weight 39 kg, goal “Muscle gain” and intensity “Medium”. The resulting plan is therefore an example of how the same application can produce output based on selected user attributes rather than displaying a fixed generic schedule.

13.3 Application Execution

The terminal evidence shows Uvicorn running on the local address and receiving successful requests. The Swagger UI also confirms that the application exposes documented API routes for its major functions.

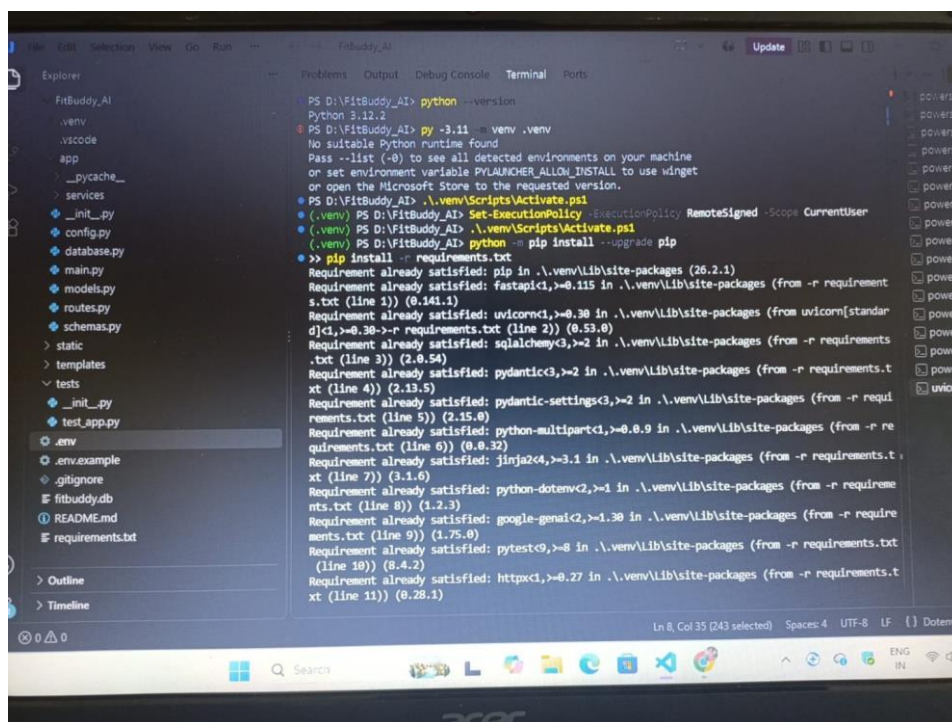


Figure 13.1: FitBuddy generated plan output page.

13.4 Project Outcome

Overall, the project demonstrates an end-to-end AI web application in which user input is collected through a browser, processed through FastAPI, sent to Gemini-based generation functions, stored in SQLite and rendered back through Jinja2 templates.